

Week 1 — C# + OOP + Git

Trainee Training Plan (8-Week Program) · Week 1 of 8 · TicketDesk Project · 5 working days

End-of-week deliverable: A console app containing `Ticket` and `User`, pushed to a repo via a branch + Pull Request. Goal: the trainee learns C# and OOP basics and works with Git from day one.

⚠️ **Note:** The `Ticket` / `User` classes this week are **practice exercises only** — a warm-up to make the names familiar. They are **not** the real project code. The actual TicketDesk project (Entities, DbContext, API) starts in **Week 3**.

Day 1 Environment Setup + First Program

Learn:

- Install: Visual Studio (or VS Code) + .NET SDK 6 + Git + an Azure DevOps / GitHub account.
- What .NET is, and the difference between the SDK and the Runtime.
- First console app: `dotnet new console`, understand `Main` and `Console.WriteLine`.

Resources:

- Install .NET SDK: dotnet.microsoft.com/download/dotnet/6.0
- Install Git: git-scm.com/downloads
- Hello World tutorial: learn.microsoft.com/dotnet/core/tutorials

Practice:

- Run "Hello World".
- Variables: `int`, `string`, `bool`, `double` and printing them.

✅ End of day: a program that prints the trainee's name and some simple data.

Day 2

C# Fundamentals

Learn:

- Conditions: `if / else` , `switch` .
- Loops: `for` , `foreach` , `while` .
- Arrays and lists: `array` , `List<T>` .
- Methods: parameters and return values.

Resources:

- C# fundamentals: learn.microsoft.com/dotnet/csharp
- Conditions & loops: [w3schools.com/cs_\(conditions_&_loops\)](https://w3schools.com/cs_(conditions_&_loops))
- Methods: learn.microsoft.com/dotnet/csharp/methods

Practice:

- A program that stores a list of names in a `List<string>` and prints them in a loop.
- A method that takes two numbers and returns their sum.

✓ End of day: the trainee writes methods and loops without help.

Day 3

Git — The Most Practical Day

Learn:

- Why Git? The difference between a local and a remote repo.
- Commands: `clone` , `status` , `add` , `commit` , `push` , `pull` .
- Branches: `branch` , `checkout` , and the idea of a Pull Request.

Resources:

- Git handbook: guides.github.com/introduction/git-handbook
- Interactive practice: learngitbranching.js.org
- W3Schools Git: w3schools.com/git

Practice:

- Create a repo and push the last two days' code to it.
- Create a branch named `feature/hello` , then commit, push, and open the first Pull Request.

✓ End of day: the trainee does branch → commit → push → PR on their own.

💡 Keep all following days on branches & PRs — so they get used to it early.

Day 4**OOP — Classes & Objects****Learn:**

- The difference between a class and an object.
- Properties and methods.
- The constructor.
- Encapsulation: `public` / `private` .

Resources:

- C# OOP overview: tutorialsteacher.com/csharp/oop
- Classes & objects: learn.microsoft.com/dotnet/csharp/.../classes
- W3Schools C# classes: [w3schools.com/cs \(classes\)](https://w3schools.com/cs/classes).

Practice:

- A `User` class (properties: Id, Name, Email).
- A `Ticket` class (properties: Id, Title, Description, Status).
- Create objects from them and print them.

✓ End of day: classes with properties and a constructor, and working objects.

Day 5**OOP — Inheritance + Interfaces + Handover****Learn:**

- Inheritance — a class inheriting from another class.
- `interface` — the contract and its implementation.
- Polymorphism (a simple introduction).

Resources:

- Inheritance: learn.microsoft.com/dotnet/csharp/.../inheritance
- Interfaces: learn.microsoft.com/dotnet/csharp/.../interfaces
- Polymorphism: [w3schools.com/cs \(polymorphism\)](https://w3schools.com/cs/polymorphism).

Practice:

- An `IEntity` interface with an `Id` .
- `User` and `Ticket` implement `IEntity` .
- A method that prints any `IEntity` .
- Tidy up the final project, commit, and open the final PR of the week.

✓ End of week: a console app with User, Ticket, and IEntity, pushed via a clean PR.

Notes for the Trainer

- **At least 50% hands-on** — don't let a full day be just reading.
- **Quick daily review** (15 minutes at day's end): what was understood, what was hard.
- **Git is mandatory from Day 3** — any code after it goes through a branch + PR.
- If the trainee finishes early: add 2–3 LeetCode Easy problems (arrays / strings).